

10_Encontrar_Muestra

April 24, 2026

0.0.1 Cargar Dataset

```
[1]: from pathlib import Path
import pandas as pd
import time
```

```
[2]: PROJECT_ROOT = Path("/home/harielpadillasanchez/Documentos/TT/TT2")
DATA_DIR = PROJECT_ROOT / "data"

FEINA_PATH = DATA_DIR / "Dataset_FEINA.xlsx"
MUESTRA_PATH = DATA_DIR / "Muestra_csv.csv"

CLEAN_OUT_PATH = DATA_DIR / "Dataset_FEINA_clean_base.csv"
```

```
[3]: t0 = time.perf_counter()

df_feina = pd.read_excel(FEINA_PATH)
df_muestra = pd.read_csv(MUESTRA_PATH)

t1 = time.perf_counter()

print(f"Tiempo de carga: {t1 - t0:.2f} segundos")
print("Shape FEINA original:", df_feina.shape)
print("Shape muestra:", df_muestra.shape)
print("\nColumnas FEINA:")
print(df_feina.columns.tolist())
```

Tiempo de carga: 0.38 segundos
Shape FEINA original: (5313, 15)
Shape muestra: (20, 17)

Columnas FEINA:
['Unnamed: 0', 'idFinal', 'Segmento', 'Propuesta', 'idcod', 'atr0', 'atr1',
'atr2', 'atr3', 'atr4', 'atr5', 'atr6', 'atr7', 'atr8', 'lex']

```
[4]: df = df_feina.copy()

rename_map = {
```

```

    "Unnamed: 0": "row_id",
    "Segmento": "source_text",
    "Propuesta": "reference_text",
}

df = df.rename(columns=rename_map)

required_cols = ["row_id", "idFinal", "source_text", "reference_text"]
missing = [c for c in required_cols if c not in df.columns]

if missing:
    raise ValueError(f"Faltan columnas requeridas en FEINA: {missing}")

print("Shape después de renombrar:", df.shape)
display(df[required_cols].head(3))

```

Shape después de renombrar: (5313, 15)

	row_id	idFinal	source_text \	reference_text
0	0	58_LibroBAC.pdf	Como antes explicamos, las finanzas y los conc...	0 Como antes explicamos, las finanzas están sust...
1	1	60_LibroBAC.pdf	Una vez dirimidos estos asuntos, se entra de l...	1 Una vez resueltos estos asuntos, se abordan al...
2	2	62_LibroBAC.pdf	Pero aquí no termina la utilidad del libro, ya...	2 Pero aquí no termina la utilidad del libro, pu...

```

[5]: t0 = time.perf_counter()

calibracion_inicial_ids = {3690, 1948, 286, 467, 2523}

muestra_ids = set(df_muestra["id"].dropna().astype(int).tolist())

few_shot_ids = {78, 1805, 3635, 5262}

exclude_ids = set()
exclude_ids.update(muestra_ids)
exclude_ids.update(few_shot_ids)
exclude_ids.update(calibracion_inicial_ids)

t1 = time.perf_counter()

```

```

[6]: df_clean = df[~df["row_id"].astype(int).isin(exclude_ids)].copy()

save_df = df_clean.copy()
save_df.to_csv(CLEAN_OUT_PATH, index=False, encoding="utf-8-sig")

```

```
print(f"Tiempo guardando dataset limpio: {t1 - t0:.2f} segundos")
print("Archivo guardado en:", CLEAN_OUT_PATH)
```

Tiempo guardando dataset limpio: 0.00 segundos

Archivo guardado en:

/home/harielpadillasanchez/Documentos/TT/TT2/data/Dataset_FEINA_clean_base.csv

0.0.2 Division del dataset

```
[7]: from pathlib import Path
import pandas as pd
import time
from sklearn.model_selection import train_test_split
```

```
[8]: PROJECT_ROOT = Path("/home/harielpadillasanchez/Documentos/TT/TT2")
DATA_DIR = PROJECT_ROOT / "data"
SPLIT_DIR = PROJECT_ROOT / "data" / "splits"
SPLIT_DIR.mkdir(parents=True, exist_ok=True)

CLEAN_PATH = DATA_DIR / "Dataset_FEINA_clean_base.csv"

SAMPLE_FRAC = 0.30
RANDOM_STATE = 42

TRAIN_SIZE = 0.70
VAL_SIZE = 0.15
TEST_SIZE = 0.15
```

```
[9]: df_clean = pd.read_csv(CLEAN_PATH)
print("Shape dataset limpio:", df_clean.shape)
print("Columnas:", df_clean.columns.tolist())
```

Shape dataset limpio: (5286, 15)

Columnas: ['row_id', 'idFinal', 'source_text', 'reference_text', 'idcod', 'atr0', 'atr1', 'atr2', 'atr3', 'atr4', 'atr5', 'atr6', 'atr7', 'atr8', 'lex']

```
[10]: ID_COL = "row_id"

if ID_COL not in df_clean.columns:
    raise ValueError(f"No existe la columna ID_COL={ID_COL}")

print("ID_COL usada:", ID_COL)
print("IDs únicos:", df_clean[ID_COL].nunique())
print("Filas totales:", len(df_clean))

if df_clean[ID_COL].nunique() != len(df_clean):
    print("Ojo: hay ids repetidos")
else:
```

```
print("OK: un id por fila")
```

```
ID_COL usada: row_id
IDs únicos: 5286
Filas totales: 5286
OK: un id por fila
```

```
[11]: df_30 = (
        df_clean
        .sample(frac=SAMPLE_FRAC, random_state=RANDOM_STATE)
        .copy()
        .sort_values(by=ID_COL)
        .reset_index(drop=True)
    )

    print("Shape dataset limpio:", df_clean.shape)
    print("Shape subconjunto 30%:", df_30.shape)
    print("Porcentaje real:", len(df_30) / len(df_clean))
    display(df_30.head(3))
```

```
Shape dataset limpio: (5286, 15)
Shape subconjunto 30%: (1586, 15)
Porcentaje real: 0.30003783579265986
```

	row_id	idFinal	source_text \
0	8	77_LibroBAC.pdf	Para la elaboración del documento se integró u...
1	12	82_LibroBAC.pdf	Sin dinero no se puede vivir, pudiéndose afirm...
2	14	84_LibroBAC.pdf	Para la elaboración del libro se siguió un pro...

	reference_text	idcod	atr0	atr1 \
0	Se integró un equipo profesional interdiscipli...	Fiorella	15.0	19.0
1	Se puede afirmar que el ser humano, la familia...	Fiorella	19.0	5.0
2	El proceso metodológico empleado para la elabo...	Fiorella	19.0	15.0

	atr2	atr3	atr4	atr5	atr6	atr7	atr8	lex
0	4.0	5.0	NaN	NaN	NaN	NaN	NaN	0
1	NaN	NaN	NaN	NaN	NaN	NaN	NaN	0
2	4.0	5.0	NaN	NaN	NaN	NaN	NaN	0

```
[12]: assert abs(TRAIN_SIZE + VAL_SIZE + TEST_SIZE - 1.0) < 1e-9
```

```
df_train, df_temp = train_test_split(
    df_30,
    test_size=(1 - TRAIN_SIZE),
    random_state=RANDOM_STATE,
    shuffle=True,
)

# del 30% restante, mitad val y mitad test
```

```

df_val, df_test = train_test_split(
    df_temp,
    test_size=0.50,
    random_state=RANDOM_STATE,
    shuffle=True,
)

df_train = df_train.sort_values(by=ID_COL).reset_index(drop=True)
df_val = df_val.sort_values(by=ID_COL).reset_index(drop=True)
df_test = df_test.sort_values(by=ID_COL).reset_index(drop=True)

print("Train shape:", df_train.shape)
print("Val shape:", df_val.shape)
print("Test shape:", df_test.shape)

print("\nProporciones dentro del 30%:")
print("Train:", len(df_train) / len(df_30))
print("Val:", len(df_val) / len(df_30))
print("Test:", len(df_test) / len(df_30))

```

Train shape: (1110, 15)

Val shape: (238, 15)

Test shape: (238, 15)

Proporciones dentro del 30%:

Train: 0.699873896595208

Val: 0.15006305170239598

Test: 0.15006305170239598

```

[13]: train_ids = set(df_train[ID_COL].astype(int).tolist())
      val_ids = set(df_val[ID_COL].astype(int).tolist())
      test_ids = set(df_test[ID_COL].astype(int).tolist())

      overlap_train_val = train_ids & val_ids
      overlap_train_test = train_ids & test_ids
      overlap_val_test = val_ids & test_ids

      print("Traslape train-val:", len(overlap_train_val))
      print("Traslape train-test:", len(overlap_train_test))
      print("Traslape val-test:", len(overlap_val_test))

      if len(overlap_train_val) == 0 and len(overlap_train_test) == 0 and
         len(overlap_val_test) == 0:
          print("OK: no hay fuga entre splits.")
      else:
          print("Ojo: si hay traslapes entre splits.")

```

Traslape train-val: 0

Traslape train-test: 0
Traslape val-test: 0
OK: no hay fuga entre splits.

```
[14]: prefix = "feina_clean_30"

subset_path = SPLIT_DIR / f"{prefix}_subset.csv"
train_path = SPLIT_DIR / f"{prefix}_train.csv"
val_path = SPLIT_DIR / f"{prefix}_val.csv"
test_path = SPLIT_DIR / f"{prefix}_test.csv"
meta_path = SPLIT_DIR / f"{prefix}_metadata.txt"

df_30.to_csv(subset_path, index=False, encoding="utf-8-sig")
df_train.to_csv(train_path, index=False, encoding="utf-8-sig")
df_val.to_csv(val_path, index=False, encoding="utf-8-sig")
df_test.to_csv(test_path, index=False, encoding="utf-8-sig")

with open(meta_path, "w", encoding="utf-8") as f:
    f.write(f"Dataset limpio base: {CLEAN_PATH}\n")
    f.write(f"Random state: {RANDOM_STATE}\n")
    f.write(f"Frac subset: {SAMPLE_FRAC}\n")
    f.write(f"Train size: {TRAIN_SIZE}\n")
    f.write(f"Val size: {VAL_SIZE}\n")
    f.write(f"Test size: {TEST_SIZE}\n")
    f.write(f"Shape subset: {df_30.shape}\n")
    f.write(f"Shape train: {df_train.shape}\n")
    f.write(f"Shape val: {df_val.shape}\n")
    f.write(f"Shape test: {df_test.shape}\n")

print("Archivos guardados:")
print("-", subset_path)
print("-", train_path)
print("-", val_path)
print("-", test_path)
print("-", meta_path)
```

Archivos guardados:

```
- /home/harielpadillasanchez/Documentos/TT/TT2/data/splits/feina_clean_30_subset.csv
- /home/harielpadillasanchez/Documentos/TT/TT2/data/splits/feina_clean_30_train.csv
-
/home/harielpadillasanchez/Documentos/TT/TT2/data/splits/feina_clean_30_val.csv
-
/home/harielpadillasanchez/Documentos/TT/TT2/data/splits/feina_clean_30_test.csv
- /home/harielpadillasanchez/Documentos/TT/TT2/data/splits/feina_clean_30_metadata.txt
```

1 Resumen de Partición Experimental

Origen de los datos: * Se partió del dataset **FEINA** limpio, después de excluir: * La muestra de refinamiento. * Los ejemplos *few-shot*. * Los IDs de calibración inicial.

Configuración técnica: * **Subconjunto:** Seleccionado el 30% del dataset limpio. * **Semilla (Random State):** 42 (Garantiza reproducibilidad).

Distribución de los conjuntos: | Conjunto | Cantidad de Ejemplos | | :— | :— | | **Train** (Entrenamiento) | $\{len(df_train)\}$ | | **Validation** (Validación) | $\{len(df_val)\}$ | | **Test** (Prueba) | $\{len(df_test)\}$ |

Aplicaciones de esta partición: 1. Comparación *prompt-based*. 2. Fine-tuning mediante **LoRA**. 3. Evaluación posterior con *self-refine*.

Se consideró adecuada una partición cuando cumplía simultáneamente con: (a) ejecución estable en el hardware local disponible, (b) tiempo estimado de validación compatible con iteraciones experimentales en horas y no en días, (c) tiempo total razonable para la etapa prompt-based completa, y (d) tamaño suficiente en los subconjuntos de entrenamiento, validación y prueba para sostener las etapas posteriores de ajuste fino y refinamiento.

```
[15]: from pathlib import Path
import sys
import time
import pandas as pd
from datetime import datetime

PROJECT_ROOT = Path("/home/harielpadillasanchez/Documentos/TT/TT2")
if str(PROJECT_ROOT) not in sys.path:
    sys.path.append(str(PROJECT_ROOT))

DATA_DIR = PROJECT_ROOT / "data"
SPLIT_DIR = DATA_DIR / "splits"
OUT_DIR = PROJECT_ROOT / "outputs" / "prompt_final"
OUT_DIR.mkdir(parents=True, exist_ok=True)

PREFIX = "feina_clean_30"

TRAIN_PATH = SPLIT_DIR / f"{PREFIX}_train.csv"
VAL_PATH = SPLIT_DIR / f"{PREFIX}_val.csv"
TEST_PATH = SPLIT_DIR / f"{PREFIX}_test.csv"

t0 = time.perf_counter()

df_train = pd.read_csv(TRAIN_PATH)
df_val = pd.read_csv(VAL_PATH)
df_test = pd.read_csv(TEST_PATH)

t1 = time.perf_counter()
```

```

print(f"Tiempo cargando splits: {t1 - t0:.2f} segundos")
print("Train:", df_train.shape)
print("Val:", df_val.shape)
print("Test:", df_test.shape)

display(df_val.head(3))

```

Tiempo cargando splits: 0.01 segundos

Train: (1110, 15)

Val: (238, 15)

Test: (238, 15)

	row_id	idFinal	\
0	51	5296_LibroBAC.pdf	
1	65	5318_LibroBAC.pdf	
2	97	7624_LibroBAC.pdf	

		source_text	\
0	Por otro lado, los activos netos son los activ...		
1	Identificó que si ella quería ahorrar \$100 al ...		
2	Sé que te tomará algún tiempo hacer esto, pero...		

		reference_text	idcod	atr0	atr1	\
0	Por otro lado, los activos netos son los activ...	Fiorella	5.0	6.0		
1	Ella identificó la necesidad de recortar sus g...	Fiorella	14.0	19.0		
2	Te tomará algún tiempo hacer esto, pero este e...	Fiorella	17.0	5.0		

	atr2	atr3	atr4	atr5	atr6	atr7	atr8	lex
0	19.0	NaN	NaN	NaN	NaN	NaN	NaN	0
1	17.0	NaN	NaN	NaN	NaN	NaN	NaN	0
2	4.0	NaN	NaN	NaN	NaN	NaN	NaN	0

```

[16]: import json
from configs.models import MODELS
from src.experiment.runner import ExperimentRunner
from src.evaluation.metrics import evaluate_dataframe, summarize_metrics

```

```

/home/harielpadillasanchez/Documentos/TT/TT2/.venv-bloom/lib/python3.10/site-
packages/tqdm/auto.py:21: TqdmWarning: IProgress not found. Please update
jupyter and ipywidgets. See
https://ipywidgets.readthedocs.io/en/stable/user_install.html
from .autonotebook import tqdm as notebook_tqdm

```

```

[17]: FEW_SHOT_EXAMPLES = [
    {

```

"source": ""A continuación una tabla donde se demuestra como crecen los ahorros al pasar los años: Edad Inversión Intereses Saldo Inversión Intereses Saldo Laura Ganados Alberto Ganados 22 \$800 \$80 \$880 23 \$800 \$168 \$1.232 Valor al Jubilarse \$311.092 Valor al Jubilarse \$263.232 Menos las contribuciones iniciales \$6.400 Menos las contribuciones iniciales \$28.800 Ganancia Neta \$304.692 Ganancia Neta \$234.""",

"target": ""A continuación, presentamos una tabla donde se demuestra como crecen los ahorros al pasar los años. Las categorías en que se divide la tabla son edad, inversión, intereses ganados y saldo. La tabla contrasta las ganancias en intereses por año de Laura y Alberto con una proyección de edad de jubilación de sesenta y cinco años y según la cantidad de años que mantuvieron el ahorro.""

},
{

"source": ""El varias veces mencionado Yager, nos dice acertadamente sobre estos aspectos lo siguiente: La mayoría de ustedes están hambrientos de tener libertad financiera, están hastiados de tener batallas financieras porque son como un carrusel.""

"target": ""Yager habla acertadamente sobre estos aspectos y dice que la mayoría ambiciona tener libertad financiera y está cansada de tener batallas financieras inútiles.""

},
{

"source": ""De acuerdo con esta medición, la proporción de pobres en la población mundial quienes viven con menos de un dólar por día descendió levemente entre 1987 y 1993, pues pasó del 30% al 29%.""

"target": ""Según esta medición, la proporción de personas pobres en la población mundial descendió un poco entre mil novecientos ochenta y siete y mil novecientos noventa y tres. Es decir, el porcentaje de personas que vivían con menos de un dólar por día pasó del treinta al veintinueve por ciento.""

},
{

"source": ""- Rentabilidad

INTERÉS DE 2 PESOS SOBRE UNA INVERSIÓN DE 100 PESOS = RENTABILIDAD DE 2 % ((2 ÷ 100) × 100 = 2%) POR CADA 100 PESOS SE GANAN 2 PESOS

Endeudamiento

DEUDA DE 30 PESOS POR SOBRE EL CAPITAL DE 20 PESOS = ENDEUDAMIENTO DE 1,5 VECES EL CAPITAL (30 ÷ 20 = 1,5) POR CADA PESO DE CAPITAL EXISTE 1,5 PESOS DE DEUDA

Liquidez

80 PESOS DE DINERO DISPONIBLE POR SOBRE DEUDAS DE 40 PESOS = LIQUIDEZ DE DOS VECES EL VALOR DE LA DEUDA (80 ÷ 40 = 2) POR CADA PESO DE DEUDA SE DISPONE DE 2 PESOS PARA PAGO""

```

        "target": ""EL INTERÉS DE DOS PESOS SOBRE UNA INVERSIÓN DE CIEN PESOS_
        ↪DA UNA RENTABILIDAD DE DOS POR CIENTO. ES DECIR, POR CADA CIEN PESOS GANAMOS_
        ↪DOS PESOS. LA DEUDA DE TREINTA PESOS SOBRE EL CAPITAL DE VEINTE PESOS_
        ↪RESULTA EN UNA DEUDA DE UNO PUNTO CINCO VECES EL CAPITAL. ES DECIR, POR CADA_
        ↪PESO DE CAPITAL HAY UNO PUNTO CINCO PESOS DE DEUDA. EN OCHENTA PESOS DE_
        ↪DINERO DISPONIBLE SOBRE DEUDAS DE CUARENTA PESOS DA UNA LIQUIDEZ DEL DOBLE_
        ↪DE LA DEUDA. POR CADA PESO ADEUDADO HAY DOS PESOS PARA PAGAR.""
    },
]

```

```
FEW_SHOT_EXAMPLE_IDS = [78, 1805, 3635, 5262]
```

```
PROMPT_TYPE = "few-shot"
```

```

CANDIDATE_CONFIGS = [
    {
        "model_key": "llama3",
        "config_label": "llama3_cfg_3",
        "ruleset": "R1",
        "temperature": 0.3,
        "top_p": 0.90,
        "repetition_penalty": 1.15,
        "max_new_tokens": 256,
    },
    {
        "model_key": "llama3",
        "config_label": "llama3_cfg_2",
        "ruleset": "R2",
        "temperature": 0.7,
        "top_p": 0.90,
        "repetition_penalty": 1.10,
        "max_new_tokens": 512.0,
    },
    {
        "model_key": "mistral",
        "config_label": "mistral_cfg_3",
        "ruleset": "R0",
        "temperature": 0.2,
        "top_p": 0.85,
        "repetition_penalty": 1.10,
        "max_new_tokens": 256,
    },
    {
        "model_key": "mistral",
        "config_label": "mistral_cfg_1",
        "ruleset": "R4",
        "temperature": 0.3,
        "top_p": 0.90,
    }
]

```

```

        "repetition_penalty": 1.10,
        "max_new_tokens": 256,
    },
]

display(pd.DataFrame(CANDIDATE_CONFIGS))

```

	model_key	config_label	ruleset	temperature	top_p	repetition_penalty \
0	llama3	llama3_cfg_3	R1	0.3	0.90	1.15
1	llama3	llama3_cfg_2	R2	0.7	0.90	1.10
2	mistral	mistral_cfg_3	R0	0.2	0.85	1.10
3	mistral	mistral_cfg_1	R4	0.3	0.90	1.10

	max_new_tokens
0	256.0
1	512.0
2	256.0
3	256.0

```

[18]: def run_prompt_eval_on_split(
    df_split: pd.DataFrame,
    split_name: str,
    candidate_configs: list,
    prompt_type: str,
    few_shot_examples: list | None,
    few_shot_example_ids: list | None,
    output_dir: Path,
):
    run_ts = datetime.now().strftime("%Y%m%d_%H%M%S")
    experiment_id = f"exp_prompt_final_{split_name}_{run_ts}"

    runner = ExperimentRunner(
        experiment_id=experiment_id,
        log_dir=str(PROJECT_ROOT / "outputs" / "logs")
    )

    records = []
    total_runs = len(df_split) * len(candidate_configs)
    run_counter = 0

    t0 = time.perf_counter()

    for cfg in candidate_configs:
        for _, row in df_split.iterrows():
            run_counter += 1
            print(
                f"[{run_counter}/{total_runs}] "

```

```

        f"split={split_name} | model={cfg['model_key']} | "
        f"cfg={cfg['config_label']} | ruleset={cfg['ruleset']} | "
        f"sample_id={row['row_id']}"
    )

    record = runner.run_one(
        dataset_name=f"feina_clean_30_{split_name}",
        model_key=cfg["model_key"],
        prompt_type=prompt_type,
        ruleset=cfg["ruleset"],
        source_text=row["source_text"],
        reference_text=row["reference_text"],
        sample_id=str(row["row_id"]),
        fold_id=None,
        split_name=split_name,
        few_shot_examples=few_shot_examples if prompt_type == "
↪ "few-shot" else None,
        few_shot_example_ids=few_shot_example_ids if prompt_type == "
↪ "few-shot" else None,
        generation_config={
            "temperature": cfg["temperature"],
            "top_p": cfg["top_p"],
            "repetition_penalty": cfg["repetition_penalty"],
            "max_new_tokens": cfg["max_new_tokens"],
        },
    )

    record_dict = record.to_dict()
    record_dict["config_label"] = cfg["config_label"]

    # arrastrar metadata útil si existe
    for meta_col in ["idFinal", "idcod", "lex", "atr0", "atr1", "atr2", "
↪ atr3", "atr4", "atr5", "atr6", "atr7", "atr8"]:
        if meta_col in row.index:
            record_dict[meta_col] = row[meta_col]

    records.append(record_dict)

    t1 = time.perf_counter()
    elapsed = t1 - t0

    raw_df = pd.DataFrame(records)
    raw_path = output_dir / f"{experiment_id}_raw_results.csv"
    raw_df.to_csv(raw_path, index=False, encoding="utf-8-sig")

    success_df = raw_df[raw_df["status"] == "success"].copy()

```

```

eval_t0 = time.perf_counter()
evaluated_df = evaluate_dataframe(
    success_df,
    source_col="source_text",
    pred_col="generated_text",
    ref_col="reference_text",
)
eval_t1 = time.perf_counter()

param_cols = evaluated_df["generation_config"].apply(pd.Series)
param_cols = param_cols.rename(columns=lambda c: f"gen_{c}")
evaluated_df = pd.concat([evaluated_df, param_cols], axis=1)

evaluated_path = output_dir / f"{experiment_id}_evaluated.csv"
evaluated_df.to_csv(evaluated_path, index=False, encoding="utf-8-sig")

summary_df = summarize_metrics(
    evaluated_df,
    group_cols=[
        "model_key",
        "config_label",
        "ruleset",
        "gen_temperature",
        "gen_top_p",
        "gen_repetition_penalty",
        "gen_max_new_tokens",
    ],
)
summary_path = output_dir / f"{experiment_id}_summary.csv"
summary_df.to_csv(summary_path, index=False, encoding="utf-8-sig")

meta = {
    "experiment_id": experiment_id,
    "split_name": split_name,
    "n_rows_split": int(len(df_split)),
    "n_candidate_configs": int(len(candidate_configs)),
    "expected_total_runs": int(total_runs),
    "successful_runs": int(len(success_df)),
    "prompt_type": prompt_type,
    "few_shot_example_ids": few_shot_example_ids,
    "runtime_inference_seconds": round(elapsed, 4),
    "runtime_evaluation_seconds": round(eval_t1 - eval_t0, 4),
    "raw_path": str(raw_path),
    "evaluated_path": str(evaluated_path),
    "summary_path": str(summary_path),
}

```

```

meta_path = output_dir / f"{experiment_id}_metadata.json"
with open(meta_path, "w", encoding="utf-8") as f:
    json.dump(meta, f, ensure_ascii=False, indent=2)

print("\nResumen del split")
print("-" * 50)
print("experiment_id:", experiment_id)
print("corridas esperadas:", total_runs)
print("corridas exitosas:", len(success_df))
print("tiempo inferencia (s):", round(elapsed, 2))
print("tiempo evaluación (s):", round(eval_t1 - eval_t0, 2))
print("raw_path:", raw_path)
print("evaluated_path:", evaluated_path)
print("summary_path:", summary_path)

return {
    "experiment_id": experiment_id,
    "raw_df": raw_df,
    "evaluated_df": evaluated_df,
    "summary_df": summary_df,
    "metadata": meta,
}

```

```

[19]: BENCHMARK_VAL_N = 10
RANDOM_STATE = 42
ID_COL = "row_id"

if len(df_val) < BENCHMARK_VAL_N:
    df_val_bench = df_val.copy()
else:
    df_val_bench = (
        df_val
        .sample(n=BENCHMARK_VAL_N, random_state=RANDOM_STATE)
        .copy()
        .sort_values(by=ID_COL)
        .reset_index(drop=True)
    )

print("Shape val total:", df_val.shape)
print("Shape val benchmark:", df_val_bench.shape)
display(df_val_bench[[ID_COL, "source_text", "reference_text"]].head(3))

```

Shape val total: (238, 15)

Shape val benchmark: (10, 15)

	row_id	source_text \
0	168	De igual manera se presenta el concepto de las...
1	195	Es un tipo de financiamiento a corto plazo que...

2 317 Su vida y las de sus seres queridos, dice el I...

reference_text

0 Igualmente, se presenta el concepto de las tar...

1 Es un tipo de financiamiento a corto plazo obt...

2 El Instituto Nacional de Seguros dice que su v...

```
[20]: benchmark_results_30 = run_prompt_eval_on_split(  
      df_split=df_val_bench,  
      split_name="val_benchmark_30",  
      candidate_configs=CANDIDATE_CONFIGS,  
      prompt_type=PROMPT_TYPE,  
      few_shot_examples=FEW_SHOT_EXAMPLES,  
      few_shot_example_ids=FEW_SHOT_EXAMPLE_IDS,  
      output_dir=OUT_DIR,  
      )
```

[1/40] split=val_benchmark_30 | model=llama3 | cfg=llama3_cfg_3 | ruleset=R1 |
sample_id=168

[2/40] split=val_benchmark_30 | model=llama3 | cfg=llama3_cfg_3 | ruleset=R1 |
sample_id=195

[3/40] split=val_benchmark_30 | model=llama3 | cfg=llama3_cfg_3 | ruleset=R1 |
sample_id=317

[4/40] split=val_benchmark_30 | model=llama3 | cfg=llama3_cfg_3 | ruleset=R1 |
sample_id=2562

[5/40] split=val_benchmark_30 | model=llama3 | cfg=llama3_cfg_3 | ruleset=R1 |
sample_id=2650

[6/40] split=val_benchmark_30 | model=llama3 | cfg=llama3_cfg_3 | ruleset=R1 |
sample_id=2841

[7/40] split=val_benchmark_30 | model=llama3 | cfg=llama3_cfg_3 | ruleset=R1 |
sample_id=4029

[8/40] split=val_benchmark_30 | model=llama3 | cfg=llama3_cfg_3 | ruleset=R1 |
sample_id=4818

[9/40] split=val_benchmark_30 | model=llama3 | cfg=llama3_cfg_3 | ruleset=R1 |
sample_id=4973

[10/40] split=val_benchmark_30 | model=llama3 | cfg=llama3_cfg_3 | ruleset=R1 |
sample_id=5019

[11/40] split=val_benchmark_30 | model=llama3 | cfg=llama3_cfg_2 | ruleset=R2 |
sample_id=168

[12/40] split=val_benchmark_30 | model=llama3 | cfg=llama3_cfg_2 | ruleset=R2 |
sample_id=195

[13/40] split=val_benchmark_30 | model=llama3 | cfg=llama3_cfg_2 | ruleset=R2 |
sample_id=317

[14/40] split=val_benchmark_30 | model=llama3 | cfg=llama3_cfg_2 | ruleset=R2 |
sample_id=2562

[15/40] split=val_benchmark_30 | model=llama3 | cfg=llama3_cfg_2 | ruleset=R2 |
sample_id=2650

[16/40] split=val_benchmark_30 | model=llama3 | cfg=llama3_cfg_2 | ruleset=R2 |

sample_id=2841
 [17/40] split=val_benchmark_30 | model=llama3 | cfg=llama3_cfg_2 | ruleset=R2 |
 sample_id=4029
 [18/40] split=val_benchmark_30 | model=llama3 | cfg=llama3_cfg_2 | ruleset=R2 |
 sample_id=4818
 [19/40] split=val_benchmark_30 | model=llama3 | cfg=llama3_cfg_2 | ruleset=R2 |
 sample_id=4973
 [20/40] split=val_benchmark_30 | model=llama3 | cfg=llama3_cfg_2 | ruleset=R2 |
 sample_id=5019
 [21/40] split=val_benchmark_30 | model=mistral | cfg=mistral_cfg_3 | ruleset=R0
 | sample_id=168
 [22/40] split=val_benchmark_30 | model=mistral | cfg=mistral_cfg_3 | ruleset=R0
 | sample_id=195
 [23/40] split=val_benchmark_30 | model=mistral | cfg=mistral_cfg_3 | ruleset=R0
 | sample_id=317
 [24/40] split=val_benchmark_30 | model=mistral | cfg=mistral_cfg_3 | ruleset=R0
 | sample_id=2562
 [25/40] split=val_benchmark_30 | model=mistral | cfg=mistral_cfg_3 | ruleset=R0
 | sample_id=2650
 [26/40] split=val_benchmark_30 | model=mistral | cfg=mistral_cfg_3 | ruleset=R0
 | sample_id=2841
 [27/40] split=val_benchmark_30 | model=mistral | cfg=mistral_cfg_3 | ruleset=R0
 | sample_id=4029
 [28/40] split=val_benchmark_30 | model=mistral | cfg=mistral_cfg_3 | ruleset=R0
 | sample_id=4818
 [29/40] split=val_benchmark_30 | model=mistral | cfg=mistral_cfg_3 | ruleset=R0
 | sample_id=4973
 [30/40] split=val_benchmark_30 | model=mistral | cfg=mistral_cfg_3 | ruleset=R0
 | sample_id=5019
 [31/40] split=val_benchmark_30 | model=mistral | cfg=mistral_cfg_1 | ruleset=R4
 | sample_id=168
 [32/40] split=val_benchmark_30 | model=mistral | cfg=mistral_cfg_1 | ruleset=R4
 | sample_id=195
 [33/40] split=val_benchmark_30 | model=mistral | cfg=mistral_cfg_1 | ruleset=R4
 | sample_id=317
 [34/40] split=val_benchmark_30 | model=mistral | cfg=mistral_cfg_1 | ruleset=R4
 | sample_id=2562
 [35/40] split=val_benchmark_30 | model=mistral | cfg=mistral_cfg_1 | ruleset=R4
 | sample_id=2650
 [36/40] split=val_benchmark_30 | model=mistral | cfg=mistral_cfg_1 | ruleset=R4
 | sample_id=2841
 [37/40] split=val_benchmark_30 | model=mistral | cfg=mistral_cfg_1 | ruleset=R4
 | sample_id=4029
 [38/40] split=val_benchmark_30 | model=mistral | cfg=mistral_cfg_1 | ruleset=R4
 | sample_id=4818
 [39/40] split=val_benchmark_30 | model=mistral | cfg=mistral_cfg_1 | ruleset=R4
 | sample_id=4973
 [40/40] split=val_benchmark_30 | model=mistral | cfg=mistral_cfg_1 | ruleset=R4

| sample_id=5019

Resumen del split

```
-----
experiment_id: exp_prompt_final_val_benchmark_30_20260424_012135
corridas esperadas: 40
corridas exitosas: 40
tiempo inferencia (s): 183.8
tiempo evaluación (s): 4.11
raw_path: /home/harielpadillasanchez/Documentos/TT/TT2/outputs/prompt_final/exp_
prompt_final_val_benchmark_30_20260424_012135_raw_results.csv
evaluated_path: /home/harielpadillasanchez/Documentos/TT/TT2/outputs/prompt_fina
l/exp_prompt_final_val_benchmark_30_20260424_012135_evaluated.csv
summary_path: /home/harielpadillasanchez/Documentos/TT/TT2/outputs/prompt_final/
exp_prompt_final_val_benchmark_30_20260424_012135_summary.csv
```

```
[21]: bench_meta = benchmark_results_30["metadata"]

bench_n_rows = bench_meta["n_rows_split"]
bench_n_configs = bench_meta["n_candidate_configs"]
bench_total_runs = bench_meta["expected_total_runs"]
bench_inference_seconds = bench_meta["runtime_inference_seconds"]

seconds_per_run = bench_inference_seconds / bench_total_runs
estimated_full_val_runs = len(df_val) * len(CANDIDATE_CONFIGS)
estimated_full_val_seconds = seconds_per_run * estimated_full_val_runs

print("Resumen benchmark 30%")
print("-" * 50)
print("Filas benchmark:", bench_n_rows)
print("Configs benchmark:", bench_n_configs)
print("Corridas benchmark:", bench_total_runs)
print("Tiempo inferencia benchmark (s):", round(bench_inference_seconds, 2))
print("Tiempo promedio por corrida (s):", round(seconds_per_run, 4))
print("Corridas estimadas para val completo:", estimated_full_val_runs)
print("Tiempo estimado para val completo (s):",
      round(estimated_full_val_seconds, 2))
print("Tiempo estimado para val completo (min):",
      round(estimated_full_val_seconds / 60, 2))
print("Tiempo estimado para val completo (hrs):",
      round(estimated_full_val_seconds / 3600, 2))
```

Resumen benchmark 30%

```
-----
Filas benchmark: 10
Configs benchmark: 4
Corridas benchmark: 40
Tiempo inferencia benchmark (s): 183.8
```

Tiempo promedio por corrida (s): 4.595
Corridas estimadas para val completo: 952
Tiempo estimado para val completo (s): 4374.42
Tiempo estimado para val completo (min): 72.91
Tiempo estimado para val completo (hrs): 1.22

```
[22]: import json
      from datetime import datetime

      benchmark_summary = {
          "timestamp": datetime.now().isoformat(),
          "partition_name": "feina_clean_30",
          "val_total_rows": int(len(df_val)),
          "benchmark_rows": int(len(df_val_bench)),
          "n_candidate_configs": int(len(CANDIDATE_CONFIGS)),
          "benchmark_total_runs": int(bench_total_runs),
          "benchmark_inference_seconds": float(bench_inference_seconds),
          "seconds_per_run": float(seconds_per_run),
          "estimated_full_val_runs": int(estimated_full_val_runs),
          "estimated_full_val_seconds": float(estimated_full_val_seconds),
          "estimated_full_val_minutes": float(estimated_full_val_seconds / 60),
          "estimated_full_val_hours": float(estimated_full_val_seconds / 3600),
      }

      benchmark_meta_path = OUT_DIR / "benchmark_30_summary.json"
      with open(benchmark_meta_path, "w", encoding="utf-8") as f:
          json.dump(benchmark_summary, f, ensure_ascii=False, indent=2)

      print("Bitácora benchmark guardada en:")
      print(benchmark_meta_path)
```

Bitácora benchmark guardada en:
/home/harielpadillasanchez/Documentos/TT/TT2/outputs/prompt_final/benchmark_30_summary.json

```
[23]: from sklearn.model_selection import train_test_split

      TRAIN_SIZE = 0.70
      VAL_SIZE = 0.15
      TEST_SIZE = 0.15
      RANDOM_STATE = 42
      ID_COL = "row_id"

      def build_and_save_partition(df_clean, sample_frac, split_dir,
          prefix_base="feina_clean"):
          assert abs(TRAIN_SIZE + VAL_SIZE + TEST_SIZE - 1.0) < 1e-9

          t0 = time.perf_counter()
```

```

df_subset = (
    df_clean
    .sample(frac=sample_frac, random_state=RANDOM_STATE)
    .copy()
    .sort_values(by=ID_COL)
    .reset_index(drop=True)
)

df_train, df_temp = train_test_split(
    df_subset,
    test_size=(1 - TRAIN_SIZE),
    random_state=RANDOM_STATE,
    shuffle=True,
)

df_val, df_test = train_test_split(
    df_temp,
    test_size=0.50,
    random_state=RANDOM_STATE,
    shuffle=True,
)

df_train = df_train.sort_values(by=ID_COL).reset_index(drop=True)
df_val = df_val.sort_values(by=ID_COL).reset_index(drop=True)
df_test = df_test.sort_values(by=ID_COL).reset_index(drop=True)

# Verificación de traslapes
train_ids = set(df_train[ID_COL].astype(int).tolist())
val_ids = set(df_val[ID_COL].astype(int).tolist())
test_ids = set(df_test[ID_COL].astype(int).tolist())

overlap_train_val = train_ids & val_ids
overlap_train_test = train_ids & test_ids
overlap_val_test = val_ids & test_ids

t1 = time.perf_counter()

percent_label = int(sample_frac * 100)
prefix = f"{prefix_base}_{percent_label}"

subset_path = split_dir / f"{prefix}_subset.csv"
train_path = split_dir / f"{prefix}_train.csv"
val_path = split_dir / f"{prefix}_val.csv"
test_path = split_dir / f"{prefix}_test.csv"
meta_path = split_dir / f"{prefix}_metadata.txt"

```

```

df_subset.to_csv(subset_path, index=False, encoding="utf-8-sig")
df_train.to_csv(train_path, index=False, encoding="utf-8-sig")
df_val.to_csv(val_path, index=False, encoding="utf-8-sig")
df_test.to_csv(test_path, index=False, encoding="utf-8-sig")

with open(meta_path, "w", encoding="utf-8") as f:
    f.write(f"Random state: {RANDOM_STATE}\n")
    f.write(f"Frac subset: {sample_frac}\n")
    f.write(f"Train size: {TRAIN_SIZE}\n")
    f.write(f"Val size: {VAL_SIZE}\n")
    f.write(f"Test size: {TEST_SIZE}\n")
    f.write(f"Shape subset: {df_subset.shape}\n")
    f.write(f"Shape train: {df_train.shape}\n")
    f.write(f"Shape val: {df_val.shape}\n")
    f.write(f"Shape test: {df_test.shape}\n")
    f.write(f"Overlap train-val: {len(overlap_train_val)}\n")
    f.write(f"Overlap train-test: {len(overlap_train_test)}\n")
    f.write(f"Overlap val-test: {len(overlap_val_test)}\n")
    f.write(f"Build seconds: {round(t1 - t0, 4)}\n")

return {
    "sample_frac": sample_frac,
    "percent_label": percent_label,
    "subset": df_subset,
    "train": df_train,
    "val": df_val,
    "test": df_test,
    "subset_path": subset_path,
    "train_path": train_path,
    "val_path": val_path,
    "test_path": test_path,
    "meta_path": meta_path,
    "build_seconds": round(t1 - t0, 4),
    "overlap_train_val": len(overlap_train_val),
    "overlap_train_test": len(overlap_train_test),
    "overlap_val_test": len(overlap_val_test),
}

```

```

[24]: partition_20 = build_and_save_partition(
    df_clean=df_clean,
    sample_frac=0.20,
    split_dir=SPLIT_DIR,
    prefix_base="feina_clean"
)

partition_40 = build_and_save_partition(
    df_clean=df_clean,

```

```

    sample_frac=0.40,
    split_dir=SPLIT_DIR,
    prefix_base="feina_clean"
)

for part in [partition_20, partition_40]:
    print("-" * 60)
    print(f"Partición {part['percent_label']}%")
    print("subset:", part["subset"].shape)
    print("train :", part["train"].shape)
    print("val   :", part["val"].shape)
    print("test  :", part["test"].shape)
    print("tiempo construcción (s):", part["build_seconds"])
    print("overlap train-val :", part["overlap_train_val"])
    print("overlap train-test:", part["overlap_train_test"])
    print("overlap val-test  :", part["overlap_val_test"])
    print("meta:", part["meta_path"])

```

```

-----
Partición 20%
subset: (1057, 15)
train : (739, 15)
val   : (159, 15)
test  : (159, 15)
tiempo construcción (s): 0.0289
overlap train-val : 0
overlap train-test: 0
overlap val-test  : 0
meta: /home/harielpadillasanchez/Documentos/TT/TT2/data/splits/feina_clean_20_me
tadata.txt
-----

```

```

Partición 40%
subset: (2114, 15)
train : (1479, 15)
val   : (317, 15)
test  : (318, 15)
tiempo construcción (s): 0.0032
overlap train-val : 0
overlap train-test: 0
overlap val-test  : 0
meta: /home/harielpadillasanchez/Documentos/TT/TT2/data/splits/feina_clean_40_me
tadata.txt

```

```

[25]: def make_val_benchmark(df_val, bench_n=10, random_state=42, id_col="row_id"):
        if len(df_val) < bench_n:
            bench_df = df_val.copy()
        else:

```

```

    bench_df = (
        df_val
        .sample(n=bench_n, random_state=random_state)
        .copy()
        .sort_values(by=id_col)
        .reset_index(drop=True)
    )
    return bench_df

```

```

[26]: benchmarks = {
    "20_10": make_val_benchmark(partition_20["val"], bench_n=10,
    ↪random_state=42, id_col=ID_COL),
    "20_20": make_val_benchmark(partition_20["val"], bench_n=20,
    ↪random_state=42, id_col=ID_COL),
    "30_10": make_val_benchmark(df_val, bench_n=10,
    ↪random_state=42, id_col=ID_COL),
    "30_20": make_val_benchmark(df_val, bench_n=20,
    ↪random_state=42, id_col=ID_COL),
    "40_10": make_val_benchmark(partition_40["val"], bench_n=10,
    ↪random_state=42, id_col=ID_COL),
    "40_20": make_val_benchmark(partition_40["val"], bench_n=20,
    ↪random_state=42, id_col=ID_COL),
}

for k, v in benchmarks.items():
    print(k, v.shape)

```

```

20_10 (10, 15)
20_20 (20, 15)
30_10 (10, 15)
30_20 (20, 15)
40_10 (10, 15)
40_20 (20, 15)

```

```

[27]: benchmark_results = {}

for bench_name in ["20_10", "20_20", "40_10", "40_20"]:
    print("\n" + "=" * 70)
    print(f"Corriendo benchmark {bench_name}")
    print("=" * 70)

    bench_df = benchmarks[bench_name]

    result = run_prompt_eval_on_split(
        df_split=bench_df,
        split_name=f"val_benchmark_{bench_name}",
        candidate_configs=CANDIDATE_CONFIGS,

```

```

        prompt_type=PROMPT_TYPE,
        few_shot_examples=FEW_SHOT_EXAMPLES,
        few_shot_example_ids=FEW_SHOT_EXAMPLE_IDS,
        output_dir=OUT_DIR,
    )

    benchmark_results[bench_name] = result

```

```

=====
Corriendo benchmark 20_10
=====

```

```

[1/40] split=val_benchmark_20_10 | model=llama3 | cfg=llama3_cfg_3 | ruleset=R1
| sample_id=1016
[2/40] split=val_benchmark_20_10 | model=llama3 | cfg=llama3_cfg_3 | ruleset=R1
| sample_id=1563
[3/40] split=val_benchmark_20_10 | model=llama3 | cfg=llama3_cfg_3 | ruleset=R1
| sample_id=1658
[4/40] split=val_benchmark_20_10 | model=llama3 | cfg=llama3_cfg_3 | ruleset=R1
| sample_id=2428
[5/40] split=val_benchmark_20_10 | model=llama3 | cfg=llama3_cfg_3 | ruleset=R1
| sample_id=2937
[6/40] split=val_benchmark_20_10 | model=llama3 | cfg=llama3_cfg_3 | ruleset=R1
| sample_id=3262
[7/40] split=val_benchmark_20_10 | model=llama3 | cfg=llama3_cfg_3 | ruleset=R1
| sample_id=4243
[8/40] split=val_benchmark_20_10 | model=llama3 | cfg=llama3_cfg_3 | ruleset=R1
| sample_id=4576
[9/40] split=val_benchmark_20_10 | model=llama3 | cfg=llama3_cfg_3 | ruleset=R1
| sample_id=4726
[10/40] split=val_benchmark_20_10 | model=llama3 | cfg=llama3_cfg_3 | ruleset=R1
| sample_id=5014
[11/40] split=val_benchmark_20_10 | model=llama3 | cfg=llama3_cfg_2 | ruleset=R2
| sample_id=1016
[12/40] split=val_benchmark_20_10 | model=llama3 | cfg=llama3_cfg_2 | ruleset=R2
| sample_id=1563
[13/40] split=val_benchmark_20_10 | model=llama3 | cfg=llama3_cfg_2 | ruleset=R2
| sample_id=1658
[14/40] split=val_benchmark_20_10 | model=llama3 | cfg=llama3_cfg_2 | ruleset=R2
| sample_id=2428
[15/40] split=val_benchmark_20_10 | model=llama3 | cfg=llama3_cfg_2 | ruleset=R2
| sample_id=2937
[16/40] split=val_benchmark_20_10 | model=llama3 | cfg=llama3_cfg_2 | ruleset=R2
| sample_id=3262
[17/40] split=val_benchmark_20_10 | model=llama3 | cfg=llama3_cfg_2 | ruleset=R2
| sample_id=4243
[18/40] split=val_benchmark_20_10 | model=llama3 | cfg=llama3_cfg_2 | ruleset=R2
| sample_id=4576

```

[19/40] split=val_benchmark_20_10 | model=llama3 | cfg=llama3_cfg_2 | ruleset=R2
 | sample_id=4726
 [20/40] split=val_benchmark_20_10 | model=llama3 | cfg=llama3_cfg_2 | ruleset=R2
 | sample_id=5014
 [21/40] split=val_benchmark_20_10 | model=mistral | cfg=mistral_cfg_3 |
 ruleset=R0 | sample_id=1016
 [22/40] split=val_benchmark_20_10 | model=mistral | cfg=mistral_cfg_3 |
 ruleset=R0 | sample_id=1563
 [23/40] split=val_benchmark_20_10 | model=mistral | cfg=mistral_cfg_3 |
 ruleset=R0 | sample_id=1658
 [24/40] split=val_benchmark_20_10 | model=mistral | cfg=mistral_cfg_3 |
 ruleset=R0 | sample_id=2428
 [25/40] split=val_benchmark_20_10 | model=mistral | cfg=mistral_cfg_3 |
 ruleset=R0 | sample_id=2937
 [26/40] split=val_benchmark_20_10 | model=mistral | cfg=mistral_cfg_3 |
 ruleset=R0 | sample_id=3262
 [27/40] split=val_benchmark_20_10 | model=mistral | cfg=mistral_cfg_3 |
 ruleset=R0 | sample_id=4243
 [28/40] split=val_benchmark_20_10 | model=mistral | cfg=mistral_cfg_3 |
 ruleset=R0 | sample_id=4576
 [29/40] split=val_benchmark_20_10 | model=mistral | cfg=mistral_cfg_3 |
 ruleset=R0 | sample_id=4726
 [30/40] split=val_benchmark_20_10 | model=mistral | cfg=mistral_cfg_3 |
 ruleset=R0 | sample_id=5014
 [31/40] split=val_benchmark_20_10 | model=mistral | cfg=mistral_cfg_1 |
 ruleset=R4 | sample_id=1016
 [32/40] split=val_benchmark_20_10 | model=mistral | cfg=mistral_cfg_1 |
 ruleset=R4 | sample_id=1563
 [33/40] split=val_benchmark_20_10 | model=mistral | cfg=mistral_cfg_1 |
 ruleset=R4 | sample_id=1658
 [34/40] split=val_benchmark_20_10 | model=mistral | cfg=mistral_cfg_1 |
 ruleset=R4 | sample_id=2428
 [35/40] split=val_benchmark_20_10 | model=mistral | cfg=mistral_cfg_1 |
 ruleset=R4 | sample_id=2937
 [36/40] split=val_benchmark_20_10 | model=mistral | cfg=mistral_cfg_1 |
 ruleset=R4 | sample_id=3262
 [37/40] split=val_benchmark_20_10 | model=mistral | cfg=mistral_cfg_1 |
 ruleset=R4 | sample_id=4243
 [38/40] split=val_benchmark_20_10 | model=mistral | cfg=mistral_cfg_1 |
 ruleset=R4 | sample_id=4576
 [39/40] split=val_benchmark_20_10 | model=mistral | cfg=mistral_cfg_1 |
 ruleset=R4 | sample_id=4726
 [40/40] split=val_benchmark_20_10 | model=mistral | cfg=mistral_cfg_1 |
 ruleset=R4 | sample_id=5014

Resumen del split

 experiment_id: exp_prompt_final_val_benchmark_20_10_20260424_012443

corridas esperadas: 40
corridas exitosas: 40
tiempo inferencia (s): 183.7
tiempo evaluación (s): 2.58
raw_path: /home/harielpadillasanchez/Documentos/TT/TT2/outputs/prompt_final/exp_prompt_final_val_benchmark_20_10_20260424_012443_raw_results.csv
evaluated_path: /home/harielpadillasanchez/Documentos/TT/TT2/outputs/prompt_final/exp_prompt_final_val_benchmark_20_10_20260424_012443_evaluated.csv
summary_path: /home/harielpadillasanchez/Documentos/TT/TT2/outputs/prompt_final/exp_prompt_final_val_benchmark_20_10_20260424_012443_summary.csv

=====
Corriendo benchmark 20_20
=====

[1/80] split=val_benchmark_20_20 | model=llama3 | cfg=llama3_cfg_3 | ruleset=R1
| sample_id=358
[2/80] split=val_benchmark_20_20 | model=llama3 | cfg=llama3_cfg_3 | ruleset=R1
| sample_id=391
[3/80] split=val_benchmark_20_20 | model=llama3 | cfg=llama3_cfg_3 | ruleset=R1
| sample_id=476
[4/80] split=val_benchmark_20_20 | model=llama3 | cfg=llama3_cfg_3 | ruleset=R1
| sample_id=730
[5/80] split=val_benchmark_20_20 | model=llama3 | cfg=llama3_cfg_3 | ruleset=R1
| sample_id=1016
[6/80] split=val_benchmark_20_20 | model=llama3 | cfg=llama3_cfg_3 | ruleset=R1
| sample_id=1035
[7/80] split=val_benchmark_20_20 | model=llama3 | cfg=llama3_cfg_3 | ruleset=R1
| sample_id=1563
[8/80] split=val_benchmark_20_20 | model=llama3 | cfg=llama3_cfg_3 | ruleset=R1
| sample_id=1658
[9/80] split=val_benchmark_20_20 | model=llama3 | cfg=llama3_cfg_3 | ruleset=R1
| sample_id=1758
[10/80] split=val_benchmark_20_20 | model=llama3 | cfg=llama3_cfg_3 | ruleset=R1
| sample_id=1972
[11/80] split=val_benchmark_20_20 | model=llama3 | cfg=llama3_cfg_3 | ruleset=R1
| sample_id=2428
[12/80] split=val_benchmark_20_20 | model=llama3 | cfg=llama3_cfg_3 | ruleset=R1
| sample_id=2937
[13/80] split=val_benchmark_20_20 | model=llama3 | cfg=llama3_cfg_3 | ruleset=R1
| sample_id=3014
[14/80] split=val_benchmark_20_20 | model=llama3 | cfg=llama3_cfg_3 | ruleset=R1
| sample_id=3262
[15/80] split=val_benchmark_20_20 | model=llama3 | cfg=llama3_cfg_3 | ruleset=R1
| sample_id=3293
[16/80] split=val_benchmark_20_20 | model=llama3 | cfg=llama3_cfg_3 | ruleset=R1
| sample_id=4180
[17/80] split=val_benchmark_20_20 | model=llama3 | cfg=llama3_cfg_3 | ruleset=R1
| sample_id=4243

[18/80] split=val_benchmark_20_20 | model=llama3 | cfg=llama3_cfg_3 | ruleset=R1
 | sample_id=4576
 [19/80] split=val_benchmark_20_20 | model=llama3 | cfg=llama3_cfg_3 | ruleset=R1
 | sample_id=4726
 [20/80] split=val_benchmark_20_20 | model=llama3 | cfg=llama3_cfg_3 | ruleset=R1
 | sample_id=5014
 [21/80] split=val_benchmark_20_20 | model=llama3 | cfg=llama3_cfg_2 | ruleset=R2
 | sample_id=358
 [22/80] split=val_benchmark_20_20 | model=llama3 | cfg=llama3_cfg_2 | ruleset=R2
 | sample_id=391
 [23/80] split=val_benchmark_20_20 | model=llama3 | cfg=llama3_cfg_2 | ruleset=R2
 | sample_id=476
 [24/80] split=val_benchmark_20_20 | model=llama3 | cfg=llama3_cfg_2 | ruleset=R2
 | sample_id=730
 [25/80] split=val_benchmark_20_20 | model=llama3 | cfg=llama3_cfg_2 | ruleset=R2
 | sample_id=1016
 [26/80] split=val_benchmark_20_20 | model=llama3 | cfg=llama3_cfg_2 | ruleset=R2
 | sample_id=1035
 [27/80] split=val_benchmark_20_20 | model=llama3 | cfg=llama3_cfg_2 | ruleset=R2
 | sample_id=1563
 [28/80] split=val_benchmark_20_20 | model=llama3 | cfg=llama3_cfg_2 | ruleset=R2
 | sample_id=1658
 [29/80] split=val_benchmark_20_20 | model=llama3 | cfg=llama3_cfg_2 | ruleset=R2
 | sample_id=1758
 [30/80] split=val_benchmark_20_20 | model=llama3 | cfg=llama3_cfg_2 | ruleset=R2
 | sample_id=1972
 [31/80] split=val_benchmark_20_20 | model=llama3 | cfg=llama3_cfg_2 | ruleset=R2
 | sample_id=2428
 [32/80] split=val_benchmark_20_20 | model=llama3 | cfg=llama3_cfg_2 | ruleset=R2
 | sample_id=2937
 [33/80] split=val_benchmark_20_20 | model=llama3 | cfg=llama3_cfg_2 | ruleset=R2
 | sample_id=3014
 [34/80] split=val_benchmark_20_20 | model=llama3 | cfg=llama3_cfg_2 | ruleset=R2
 | sample_id=3262
 [35/80] split=val_benchmark_20_20 | model=llama3 | cfg=llama3_cfg_2 | ruleset=R2
 | sample_id=3293
 [36/80] split=val_benchmark_20_20 | model=llama3 | cfg=llama3_cfg_2 | ruleset=R2
 | sample_id=4180
 [37/80] split=val_benchmark_20_20 | model=llama3 | cfg=llama3_cfg_2 | ruleset=R2
 | sample_id=4243
 [38/80] split=val_benchmark_20_20 | model=llama3 | cfg=llama3_cfg_2 | ruleset=R2
 | sample_id=4576
 [39/80] split=val_benchmark_20_20 | model=llama3 | cfg=llama3_cfg_2 | ruleset=R2
 | sample_id=4726
 [40/80] split=val_benchmark_20_20 | model=llama3 | cfg=llama3_cfg_2 | ruleset=R2
 | sample_id=5014
 [41/80] split=val_benchmark_20_20 | model=mistral | cfg=mistral_cfg_3 |
 ruleset=R0 | sample_id=358

[42/80] split=val_benchmark_20_20 | model=mistral | cfg=mistral_cfg_3 |
 ruleset=R0 | sample_id=391
 [43/80] split=val_benchmark_20_20 | model=mistral | cfg=mistral_cfg_3 |
 ruleset=R0 | sample_id=476
 [44/80] split=val_benchmark_20_20 | model=mistral | cfg=mistral_cfg_3 |
 ruleset=R0 | sample_id=730
 [45/80] split=val_benchmark_20_20 | model=mistral | cfg=mistral_cfg_3 |
 ruleset=R0 | sample_id=1016
 [46/80] split=val_benchmark_20_20 | model=mistral | cfg=mistral_cfg_3 |
 ruleset=R0 | sample_id=1035
 [47/80] split=val_benchmark_20_20 | model=mistral | cfg=mistral_cfg_3 |
 ruleset=R0 | sample_id=1563
 [48/80] split=val_benchmark_20_20 | model=mistral | cfg=mistral_cfg_3 |
 ruleset=R0 | sample_id=1658
 [49/80] split=val_benchmark_20_20 | model=mistral | cfg=mistral_cfg_3 |
 ruleset=R0 | sample_id=1758
 [50/80] split=val_benchmark_20_20 | model=mistral | cfg=mistral_cfg_3 |
 ruleset=R0 | sample_id=1972
 [51/80] split=val_benchmark_20_20 | model=mistral | cfg=mistral_cfg_3 |
 ruleset=R0 | sample_id=2428
 [52/80] split=val_benchmark_20_20 | model=mistral | cfg=mistral_cfg_3 |
 ruleset=R0 | sample_id=2937
 [53/80] split=val_benchmark_20_20 | model=mistral | cfg=mistral_cfg_3 |
 ruleset=R0 | sample_id=3014
 [54/80] split=val_benchmark_20_20 | model=mistral | cfg=mistral_cfg_3 |
 ruleset=R0 | sample_id=3262
 [55/80] split=val_benchmark_20_20 | model=mistral | cfg=mistral_cfg_3 |
 ruleset=R0 | sample_id=3293
 [56/80] split=val_benchmark_20_20 | model=mistral | cfg=mistral_cfg_3 |
 ruleset=R0 | sample_id=4180
 [57/80] split=val_benchmark_20_20 | model=mistral | cfg=mistral_cfg_3 |
 ruleset=R0 | sample_id=4243
 [58/80] split=val_benchmark_20_20 | model=mistral | cfg=mistral_cfg_3 |
 ruleset=R0 | sample_id=4576
 [59/80] split=val_benchmark_20_20 | model=mistral | cfg=mistral_cfg_3 |
 ruleset=R0 | sample_id=4726
 [60/80] split=val_benchmark_20_20 | model=mistral | cfg=mistral_cfg_3 |
 ruleset=R0 | sample_id=5014
 [61/80] split=val_benchmark_20_20 | model=mistral | cfg=mistral_cfg_1 |
 ruleset=R4 | sample_id=358
 [62/80] split=val_benchmark_20_20 | model=mistral | cfg=mistral_cfg_1 |
 ruleset=R4 | sample_id=391
 [63/80] split=val_benchmark_20_20 | model=mistral | cfg=mistral_cfg_1 |
 ruleset=R4 | sample_id=476
 [64/80] split=val_benchmark_20_20 | model=mistral | cfg=mistral_cfg_1 |
 ruleset=R4 | sample_id=730
 [65/80] split=val_benchmark_20_20 | model=mistral | cfg=mistral_cfg_1 |
 ruleset=R4 | sample_id=1016

```

[66/80] split=val_benchmark_20_20 | model=mistral | cfg=mistral_cfg_1 |
ruleset=R4 | sample_id=1035
[67/80] split=val_benchmark_20_20 | model=mistral | cfg=mistral_cfg_1 |
ruleset=R4 | sample_id=1563
[68/80] split=val_benchmark_20_20 | model=mistral | cfg=mistral_cfg_1 |
ruleset=R4 | sample_id=1658
[69/80] split=val_benchmark_20_20 | model=mistral | cfg=mistral_cfg_1 |
ruleset=R4 | sample_id=1758
[70/80] split=val_benchmark_20_20 | model=mistral | cfg=mistral_cfg_1 |
ruleset=R4 | sample_id=1972
[71/80] split=val_benchmark_20_20 | model=mistral | cfg=mistral_cfg_1 |
ruleset=R4 | sample_id=2428
[72/80] split=val_benchmark_20_20 | model=mistral | cfg=mistral_cfg_1 |
ruleset=R4 | sample_id=2937
[73/80] split=val_benchmark_20_20 | model=mistral | cfg=mistral_cfg_1 |
ruleset=R4 | sample_id=3014
[74/80] split=val_benchmark_20_20 | model=mistral | cfg=mistral_cfg_1 |
ruleset=R4 | sample_id=3262
[75/80] split=val_benchmark_20_20 | model=mistral | cfg=mistral_cfg_1 |
ruleset=R4 | sample_id=3293
[76/80] split=val_benchmark_20_20 | model=mistral | cfg=mistral_cfg_1 |
ruleset=R4 | sample_id=4180
[77/80] split=val_benchmark_20_20 | model=mistral | cfg=mistral_cfg_1 |
ruleset=R4 | sample_id=4243
[78/80] split=val_benchmark_20_20 | model=mistral | cfg=mistral_cfg_1 |
ruleset=R4 | sample_id=4576
[79/80] split=val_benchmark_20_20 | model=mistral | cfg=mistral_cfg_1 |
ruleset=R4 | sample_id=4726
[80/80] split=val_benchmark_20_20 | model=mistral | cfg=mistral_cfg_1 |
ruleset=R4 | sample_id=5014

```

```

'(ReadTimeoutError("HTTPSConnectionPool(host='huggingface.co', port=443): Read
timed out. (read timeout=10)"), '(Request ID:
745116f7-4a19-4eaa-83ae-12c170a44d85)'))' thrown while requesting HEAD
https://huggingface.co/bert-base-multilingual-
cased/resolve/main/tokenizer_config.json
WARNING:huggingface_hub.utils._http: '(ReadTimeoutError("HTTPSConnectionPool(host
='huggingface.co', port=443): Read timed out. (read timeout=10)"), '(Request ID:
745116f7-4a19-4eaa-83ae-12c170a44d85)'))' thrown while requesting HEAD
https://huggingface.co/bert-base-multilingual-
cased/resolve/main/tokenizer_config.json
Retrying in 1s [Retry 1/5].
WARNING:huggingface_hub.utils._http:Retrying in 1s [Retry 1/5].

```

Resumen del split

```

-----
experiment_id: exp_prompt_final_val_benchmark_20_20_20260424_012749
corridas esperadas: 80

```

corridas exitosas: 80
tiempo inferencia (s): 407.48
tiempo evaluación (s): 15.86
raw_path: /home/harielpadillasanchez/Documentos/TT/TT2/outputs/prompt_final/exp_prompt_final_val_benchmark_20_20_20260424_012749_raw_results.csv
evaluated_path: /home/harielpadillasanchez/Documentos/TT/TT2/outputs/prompt_final/exp_prompt_final_val_benchmark_20_20_20260424_012749_evaluated.csv
summary_path: /home/harielpadillasanchez/Documentos/TT/TT2/outputs/prompt_final/exp_prompt_final_val_benchmark_20_20_20260424_012749_summary.csv

=====
Corriendo benchmark 40_10
=====

```
[1/40] split=val_benchmark_40_10 | model=llama3 | cfg=llama3_cfg_3 | ruleset=R1  
| sample_id=139  
[2/40] split=val_benchmark_40_10 | model=llama3 | cfg=llama3_cfg_3 | ruleset=R1  
| sample_id=419  
[3/40] split=val_benchmark_40_10 | model=llama3 | cfg=llama3_cfg_3 | ruleset=R1  
| sample_id=1016  
[4/40] split=val_benchmark_40_10 | model=llama3 | cfg=llama3_cfg_3 | ruleset=R1  
| sample_id=1179  
[5/40] split=val_benchmark_40_10 | model=llama3 | cfg=llama3_cfg_3 | ruleset=R1  
| sample_id=1703  
[6/40] split=val_benchmark_40_10 | model=llama3 | cfg=llama3_cfg_3 | ruleset=R1  
| sample_id=1955  
[7/40] split=val_benchmark_40_10 | model=llama3 | cfg=llama3_cfg_3 | ruleset=R1  
| sample_id=3183  
[8/40] split=val_benchmark_40_10 | model=llama3 | cfg=llama3_cfg_3 | ruleset=R1  
| sample_id=3357  
[9/40] split=val_benchmark_40_10 | model=llama3 | cfg=llama3_cfg_3 | ruleset=R1  
| sample_id=4307  
[10/40] split=val_benchmark_40_10 | model=llama3 | cfg=llama3_cfg_3 | ruleset=R1  
| sample_id=4641  
[11/40] split=val_benchmark_40_10 | model=llama3 | cfg=llama3_cfg_2 | ruleset=R2  
| sample_id=139  
[12/40] split=val_benchmark_40_10 | model=llama3 | cfg=llama3_cfg_2 | ruleset=R2  
| sample_id=419  
[13/40] split=val_benchmark_40_10 | model=llama3 | cfg=llama3_cfg_2 | ruleset=R2  
| sample_id=1016  
[14/40] split=val_benchmark_40_10 | model=llama3 | cfg=llama3_cfg_2 | ruleset=R2  
| sample_id=1179  
[15/40] split=val_benchmark_40_10 | model=llama3 | cfg=llama3_cfg_2 | ruleset=R2  
| sample_id=1703  
[16/40] split=val_benchmark_40_10 | model=llama3 | cfg=llama3_cfg_2 | ruleset=R2  
| sample_id=1955  
[17/40] split=val_benchmark_40_10 | model=llama3 | cfg=llama3_cfg_2 | ruleset=R2  
| sample_id=3183  
[18/40] split=val_benchmark_40_10 | model=llama3 | cfg=llama3_cfg_2 | ruleset=R2
```

```

| sample_id=3357
[19/40] split=val_benchmark_40_10 | model=llama3 | cfg=llama3_cfg_2 | ruleset=R2
| sample_id=4307
[20/40] split=val_benchmark_40_10 | model=llama3 | cfg=llama3_cfg_2 | ruleset=R2
| sample_id=4641
[21/40] split=val_benchmark_40_10 | model=mistral | cfg=mistral_cfg_3 |
ruleset=R0 | sample_id=139
[22/40] split=val_benchmark_40_10 | model=mistral | cfg=mistral_cfg_3 |
ruleset=R0 | sample_id=419
[23/40] split=val_benchmark_40_10 | model=mistral | cfg=mistral_cfg_3 |
ruleset=R0 | sample_id=1016
[24/40] split=val_benchmark_40_10 | model=mistral | cfg=mistral_cfg_3 |
ruleset=R0 | sample_id=1179
[25/40] split=val_benchmark_40_10 | model=mistral | cfg=mistral_cfg_3 |
ruleset=R0 | sample_id=1703
[26/40] split=val_benchmark_40_10 | model=mistral | cfg=mistral_cfg_3 |
ruleset=R0 | sample_id=1955
[27/40] split=val_benchmark_40_10 | model=mistral | cfg=mistral_cfg_3 |
ruleset=R0 | sample_id=3183
[28/40] split=val_benchmark_40_10 | model=mistral | cfg=mistral_cfg_3 |
ruleset=R0 | sample_id=3357
[29/40] split=val_benchmark_40_10 | model=mistral | cfg=mistral_cfg_3 |
ruleset=R0 | sample_id=4307
[30/40] split=val_benchmark_40_10 | model=mistral | cfg=mistral_cfg_3 |
ruleset=R0 | sample_id=4641
[31/40] split=val_benchmark_40_10 | model=mistral | cfg=mistral_cfg_1 |
ruleset=R4 | sample_id=139
[32/40] split=val_benchmark_40_10 | model=mistral | cfg=mistral_cfg_1 |
ruleset=R4 | sample_id=419
[33/40] split=val_benchmark_40_10 | model=mistral | cfg=mistral_cfg_1 |
ruleset=R4 | sample_id=1016
[34/40] split=val_benchmark_40_10 | model=mistral | cfg=mistral_cfg_1 |
ruleset=R4 | sample_id=1179
[35/40] split=val_benchmark_40_10 | model=mistral | cfg=mistral_cfg_1 |
ruleset=R4 | sample_id=1703
[36/40] split=val_benchmark_40_10 | model=mistral | cfg=mistral_cfg_1 |
ruleset=R4 | sample_id=1955
[37/40] split=val_benchmark_40_10 | model=mistral | cfg=mistral_cfg_1 |
ruleset=R4 | sample_id=3183
[38/40] split=val_benchmark_40_10 | model=mistral | cfg=mistral_cfg_1 |
ruleset=R4 | sample_id=3357
[39/40] split=val_benchmark_40_10 | model=mistral | cfg=mistral_cfg_1 |
ruleset=R4 | sample_id=4307
[40/40] split=val_benchmark_40_10 | model=mistral | cfg=mistral_cfg_1 |
ruleset=R4 | sample_id=4641

```

Resumen del split

experiment_id: exp_prompt_final_val_benchmark_40_10_20260424_013452
corridas esperadas: 40
corridas exitosas: 40
tiempo inferencia (s): 229.1
tiempo evaluación (s): 2.71
raw_path: /home/harielpadillasanchez/Documentos/TT/TT2/outputs/prompt_final/exp_prompt_final_val_benchmark_40_10_20260424_013452_raw_results.csv
evaluated_path: /home/harielpadillasanchez/Documentos/TT/TT2/outputs/prompt_final/exp_prompt_final_val_benchmark_40_10_20260424_013452_evaluated.csv
summary_path: /home/harielpadillasanchez/Documentos/TT/TT2/outputs/prompt_final/exp_prompt_final_val_benchmark_40_10_20260424_013452_summary.csv

=====
Corriendo benchmark 40_20
=====

[1/80] split=val_benchmark_40_20 | model=llama3 | cfg=llama3_cfg_3 | ruleset=R1
| sample_id=139
[2/80] split=val_benchmark_40_20 | model=llama3 | cfg=llama3_cfg_3 | ruleset=R1
| sample_id=419
[3/80] split=val_benchmark_40_20 | model=llama3 | cfg=llama3_cfg_3 | ruleset=R1
| sample_id=1016
[4/80] split=val_benchmark_40_20 | model=llama3 | cfg=llama3_cfg_3 | ruleset=R1
| sample_id=1179
[5/80] split=val_benchmark_40_20 | model=llama3 | cfg=llama3_cfg_3 | ruleset=R1
| sample_id=1278
[6/80] split=val_benchmark_40_20 | model=llama3 | cfg=llama3_cfg_3 | ruleset=R1
| sample_id=1408
[7/80] split=val_benchmark_40_20 | model=llama3 | cfg=llama3_cfg_3 | ruleset=R1
| sample_id=1703
[8/80] split=val_benchmark_40_20 | model=llama3 | cfg=llama3_cfg_3 | ruleset=R1
| sample_id=1955
[9/80] split=val_benchmark_40_20 | model=llama3 | cfg=llama3_cfg_3 | ruleset=R1
| sample_id=2226
[10/80] split=val_benchmark_40_20 | model=llama3 | cfg=llama3_cfg_3 | ruleset=R1
| sample_id=2911
[11/80] split=val_benchmark_40_20 | model=llama3 | cfg=llama3_cfg_3 | ruleset=R1
| sample_id=3098
[12/80] split=val_benchmark_40_20 | model=llama3 | cfg=llama3_cfg_3 | ruleset=R1
| sample_id=3183
[13/80] split=val_benchmark_40_20 | model=llama3 | cfg=llama3_cfg_3 | ruleset=R1
| sample_id=3353
[14/80] split=val_benchmark_40_20 | model=llama3 | cfg=llama3_cfg_3 | ruleset=R1
| sample_id=3357
[15/80] split=val_benchmark_40_20 | model=llama3 | cfg=llama3_cfg_3 | ruleset=R1
| sample_id=3718
[16/80] split=val_benchmark_40_20 | model=llama3 | cfg=llama3_cfg_3 | ruleset=R1
| sample_id=4061
[17/80] split=val_benchmark_40_20 | model=llama3 | cfg=llama3_cfg_3 | ruleset=R1

| sample_id=4142
 [18/80] split=val_benchmark_40_20 | model=llama3 | cfg=llama3_cfg_3 | ruleset=R1
 | sample_id=4307
 [19/80] split=val_benchmark_40_20 | model=llama3 | cfg=llama3_cfg_3 | ruleset=R1
 | sample_id=4641
 [20/80] split=val_benchmark_40_20 | model=llama3 | cfg=llama3_cfg_3 | ruleset=R1
 | sample_id=5237
 [21/80] split=val_benchmark_40_20 | model=llama3 | cfg=llama3_cfg_2 | ruleset=R2
 | sample_id=139
 [22/80] split=val_benchmark_40_20 | model=llama3 | cfg=llama3_cfg_2 | ruleset=R2
 | sample_id=419
 [23/80] split=val_benchmark_40_20 | model=llama3 | cfg=llama3_cfg_2 | ruleset=R2
 | sample_id=1016
 [24/80] split=val_benchmark_40_20 | model=llama3 | cfg=llama3_cfg_2 | ruleset=R2
 | sample_id=1179
 [25/80] split=val_benchmark_40_20 | model=llama3 | cfg=llama3_cfg_2 | ruleset=R2
 | sample_id=1278
 [26/80] split=val_benchmark_40_20 | model=llama3 | cfg=llama3_cfg_2 | ruleset=R2
 | sample_id=1408
 [27/80] split=val_benchmark_40_20 | model=llama3 | cfg=llama3_cfg_2 | ruleset=R2
 | sample_id=1703
 [28/80] split=val_benchmark_40_20 | model=llama3 | cfg=llama3_cfg_2 | ruleset=R2
 | sample_id=1955
 [29/80] split=val_benchmark_40_20 | model=llama3 | cfg=llama3_cfg_2 | ruleset=R2
 | sample_id=2226
 [30/80] split=val_benchmark_40_20 | model=llama3 | cfg=llama3_cfg_2 | ruleset=R2
 | sample_id=2911
 [31/80] split=val_benchmark_40_20 | model=llama3 | cfg=llama3_cfg_2 | ruleset=R2
 | sample_id=3098
 [32/80] split=val_benchmark_40_20 | model=llama3 | cfg=llama3_cfg_2 | ruleset=R2
 | sample_id=3183
 [33/80] split=val_benchmark_40_20 | model=llama3 | cfg=llama3_cfg_2 | ruleset=R2
 | sample_id=3353
 [34/80] split=val_benchmark_40_20 | model=llama3 | cfg=llama3_cfg_2 | ruleset=R2
 | sample_id=3357
 [35/80] split=val_benchmark_40_20 | model=llama3 | cfg=llama3_cfg_2 | ruleset=R2
 | sample_id=3718
 [36/80] split=val_benchmark_40_20 | model=llama3 | cfg=llama3_cfg_2 | ruleset=R2
 | sample_id=4061
 [37/80] split=val_benchmark_40_20 | model=llama3 | cfg=llama3_cfg_2 | ruleset=R2
 | sample_id=4142
 [38/80] split=val_benchmark_40_20 | model=llama3 | cfg=llama3_cfg_2 | ruleset=R2
 | sample_id=4307
 [39/80] split=val_benchmark_40_20 | model=llama3 | cfg=llama3_cfg_2 | ruleset=R2
 | sample_id=4641
 [40/80] split=val_benchmark_40_20 | model=llama3 | cfg=llama3_cfg_2 | ruleset=R2
 | sample_id=5237
 [41/80] split=val_benchmark_40_20 | model=mistral | cfg=mistral_cfg_3 |

```

ruleset=R0 | sample_id=139
[42/80] split=val_benchmark_40_20 | model=mistral | cfg=mistral_cfg_3 |
ruleset=R0 | sample_id=419
[43/80] split=val_benchmark_40_20 | model=mistral | cfg=mistral_cfg_3 |
ruleset=R0 | sample_id=1016
[44/80] split=val_benchmark_40_20 | model=mistral | cfg=mistral_cfg_3 |
ruleset=R0 | sample_id=1179
[45/80] split=val_benchmark_40_20 | model=mistral | cfg=mistral_cfg_3 |
ruleset=R0 | sample_id=1278
[46/80] split=val_benchmark_40_20 | model=mistral | cfg=mistral_cfg_3 |
ruleset=R0 | sample_id=1408
[47/80] split=val_benchmark_40_20 | model=mistral | cfg=mistral_cfg_3 |
ruleset=R0 | sample_id=1703
[48/80] split=val_benchmark_40_20 | model=mistral | cfg=mistral_cfg_3 |
ruleset=R0 | sample_id=1955
[49/80] split=val_benchmark_40_20 | model=mistral | cfg=mistral_cfg_3 |
ruleset=R0 | sample_id=2226
[50/80] split=val_benchmark_40_20 | model=mistral | cfg=mistral_cfg_3 |
ruleset=R0 | sample_id=2911
[51/80] split=val_benchmark_40_20 | model=mistral | cfg=mistral_cfg_3 |
ruleset=R0 | sample_id=3098
[52/80] split=val_benchmark_40_20 | model=mistral | cfg=mistral_cfg_3 |
ruleset=R0 | sample_id=3183
[53/80] split=val_benchmark_40_20 | model=mistral | cfg=mistral_cfg_3 |
ruleset=R0 | sample_id=3353
[54/80] split=val_benchmark_40_20 | model=mistral | cfg=mistral_cfg_3 |
ruleset=R0 | sample_id=3357
[55/80] split=val_benchmark_40_20 | model=mistral | cfg=mistral_cfg_3 |
ruleset=R0 | sample_id=3718
[56/80] split=val_benchmark_40_20 | model=mistral | cfg=mistral_cfg_3 |
ruleset=R0 | sample_id=4061
[57/80] split=val_benchmark_40_20 | model=mistral | cfg=mistral_cfg_3 |
ruleset=R0 | sample_id=4142
[58/80] split=val_benchmark_40_20 | model=mistral | cfg=mistral_cfg_3 |
ruleset=R0 | sample_id=4307
[59/80] split=val_benchmark_40_20 | model=mistral | cfg=mistral_cfg_3 |
ruleset=R0 | sample_id=4641
[60/80] split=val_benchmark_40_20 | model=mistral | cfg=mistral_cfg_3 |
ruleset=R0 | sample_id=5237
[61/80] split=val_benchmark_40_20 | model=mistral | cfg=mistral_cfg_1 |
ruleset=R4 | sample_id=139
[62/80] split=val_benchmark_40_20 | model=mistral | cfg=mistral_cfg_1 |
ruleset=R4 | sample_id=419
[63/80] split=val_benchmark_40_20 | model=mistral | cfg=mistral_cfg_1 |
ruleset=R4 | sample_id=1016
[64/80] split=val_benchmark_40_20 | model=mistral | cfg=mistral_cfg_1 |
ruleset=R4 | sample_id=1179
[65/80] split=val_benchmark_40_20 | model=mistral | cfg=mistral_cfg_1 |

```

```

ruleset=R4 | sample_id=1278
[66/80] split=val_benchmark_40_20 | model=mistral | cfg=mistral_cfg_1 |
ruleset=R4 | sample_id=1408
[67/80] split=val_benchmark_40_20 | model=mistral | cfg=mistral_cfg_1 |
ruleset=R4 | sample_id=1703
[68/80] split=val_benchmark_40_20 | model=mistral | cfg=mistral_cfg_1 |
ruleset=R4 | sample_id=1955
[69/80] split=val_benchmark_40_20 | model=mistral | cfg=mistral_cfg_1 |
ruleset=R4 | sample_id=2226
[70/80] split=val_benchmark_40_20 | model=mistral | cfg=mistral_cfg_1 |
ruleset=R4 | sample_id=2911
[71/80] split=val_benchmark_40_20 | model=mistral | cfg=mistral_cfg_1 |
ruleset=R4 | sample_id=3098
[72/80] split=val_benchmark_40_20 | model=mistral | cfg=mistral_cfg_1 |
ruleset=R4 | sample_id=3183
[73/80] split=val_benchmark_40_20 | model=mistral | cfg=mistral_cfg_1 |
ruleset=R4 | sample_id=3353
[74/80] split=val_benchmark_40_20 | model=mistral | cfg=mistral_cfg_1 |
ruleset=R4 | sample_id=3357
[75/80] split=val_benchmark_40_20 | model=mistral | cfg=mistral_cfg_1 |
ruleset=R4 | sample_id=3718
[76/80] split=val_benchmark_40_20 | model=mistral | cfg=mistral_cfg_1 |
ruleset=R4 | sample_id=4061
[77/80] split=val_benchmark_40_20 | model=mistral | cfg=mistral_cfg_1 |
ruleset=R4 | sample_id=4142
[78/80] split=val_benchmark_40_20 | model=mistral | cfg=mistral_cfg_1 |
ruleset=R4 | sample_id=4307
[79/80] split=val_benchmark_40_20 | model=mistral | cfg=mistral_cfg_1 |
ruleset=R4 | sample_id=4641
[80/80] split=val_benchmark_40_20 | model=mistral | cfg=mistral_cfg_1 |
ruleset=R4 | sample_id=5237

'(ReadTimeoutError("HTTPSConnectionPool(host='huggingface.co', port=443): Read
timed out. (read timeout=10)"), '(Request ID:
8a12fe54-4692-4dbd-96b1-ef27d7abd977)')' thrown while requesting HEAD
https://huggingface.co/bert-base-multilingual-
cased/resolve/main/tokenizer_config.json
WARNING:huggingface_hub.utils._http: '(ReadTimeoutError("HTTPSConnectionPool(host
='huggingface.co', port=443): Read timed out. (read timeout=10)"), '(Request ID:
8a12fe54-4692-4dbd-96b1-ef27d7abd977)')' thrown while requesting HEAD
https://huggingface.co/bert-base-multilingual-
cased/resolve/main/tokenizer_config.json
Retrying in 1s [Retry 1/5].
WARNING:huggingface_hub.utils._http:Retrying in 1s [Retry 1/5].

```

Resumen del split

experiment_id: exp_prompt_final_val_benchmark_40_20_20260424_013844

```

corridas esperadas: 80
corridas exitosas: 80
tiempo inferencia (s): 406.87
tiempo evaluación (s): 15.89
raw_path: /home/harielpadillasanchez/Documentos/TT/TT2/outputs/prompt_final/exp_
prompt_final_val_benchmark_40_20_20260424_013844_raw_results.csv
evaluated_path: /home/harielpadillasanchez/Documentos/TT/TT2/outputs/prompt_fina
l/exp_prompt_final_val_benchmark_40_20_20260424_013844_evaluated.csv
summary_path: /home/harielpadillasanchez/Documentos/TT/TT2/outputs/prompt_final/
exp_prompt_final_val_benchmark_40_20_20260424_013844_summary.csv

```

```

[28]: def estimate_partition_times(result_dict, full_val_len, full_test_len,
    ↪n_configs_val=4, n_configs_test=2):
    meta = result_dict["metadata"]

    bench_n_rows = meta["n_rows_split"]
    bench_total_runs = meta["expected_total_runs"]
    bench_inference_seconds = meta["runtime_inference_seconds"]

    seconds_per_run = bench_inference_seconds / bench_total_runs

    estimated_full_val_runs = full_val_len * n_configs_val
    estimated_full_val_seconds = seconds_per_run * estimated_full_val_runs

    estimated_full_test_runs = full_test_len * n_configs_test
    estimated_full_test_seconds = seconds_per_run * estimated_full_test_runs

    estimated_prompt_total_seconds = estimated_full_val_seconds +
    ↪estimated_full_test_seconds

    return {
        "bench_rows": bench_n_rows,
        "bench_total_runs": bench_total_runs,
        "bench_inference_seconds": bench_inference_seconds,
        "seconds_per_run": seconds_per_run,
        "estimated_full_val_runs": estimated_full_val_runs,
        "estimated_full_val_seconds": estimated_full_val_seconds,
        "estimated_full_test_runs": estimated_full_test_runs,
        "estimated_full_test_seconds": estimated_full_test_seconds,
        "estimated_prompt_total_seconds": estimated_prompt_total_seconds,
    }

est_20_10 = estimate_partition_times(
    benchmark_results["20_10"],
    full_val_len=len(partition_20["val"]),
    full_test_len=len(partition_20["test"])
)

```

```

est_20_20 = estimate_partition_times(
    benchmark_results["20_20"],
    full_val_len=len(partition_20["val"]),
    full_test_len=len(partition_20["test"])
)

est_40_10 = estimate_partition_times(
    benchmark_results["40_10"],
    full_val_len=len(partition_40["val"]),
    full_test_len=len(partition_40["test"])
)

est_40_20 = estimate_partition_times(
    benchmark_results["40_20"],
    full_val_len=len(partition_40["val"]),
    full_test_len=len(partition_40["test"])
)

print("Listo")

```

Listo

```

[29]: rows = [
    {
        "partition_pct": 20,
        "bench_size": 10,
        "val_rows": len(partition_20["val"]),
        "test_rows": len(partition_20["test"]),
        "seconds_per_run": est_20_10["seconds_per_run"],
        "est_val_hours": est_20_10["estimated_full_val_seconds"] / 3600,
        "est_test_hours": est_20_10["estimated_full_test_seconds"] / 3600,
        "est_prompt_total_hours": est_20_10["estimated_prompt_total_seconds"] / 3600,
    },
    {
        "partition_pct": 20,
        "bench_size": 20,
        "val_rows": len(partition_20["val"]),
        "test_rows": len(partition_20["test"]),
        "seconds_per_run": est_20_20["seconds_per_run"],
        "est_val_hours": est_20_20["estimated_full_val_seconds"] / 3600,
        "est_test_hours": est_20_20["estimated_full_test_seconds"] / 3600,
        "est_prompt_total_hours": est_20_20["estimated_prompt_total_seconds"] / 3600,
    },
    {

```

```

        "partition_pct": 30,
        "bench_size": 10,
        "val_rows": len(df_val),
        "test_rows": len(df_test),
        "seconds_per_run": seconds_per_run,
        "est_val_hours": estimated_full_val_seconds / 3600,
        "est_test_hours": (seconds_per_run * (len(df_test) * 2)) / 3600,
        "est_prompt_total_hours": (estimated_full_val_seconds +
↪(seconds_per_run * (len(df_test) * 2))) / 3600,
    },
    {
        "partition_pct": 40,
        "bench_size": 10,
        "val_rows": len(partition_40["val"]),
        "test_rows": len(partition_40["test"]),
        "seconds_per_run": est_40_10["seconds_per_run"],
        "est_val_hours": est_40_10["estimated_full_val_seconds"] / 3600,
        "est_test_hours": est_40_10["estimated_full_test_seconds"] / 3600,
        "est_prompt_total_hours": est_40_10["estimated_prompt_total_seconds"] /
↪3600,
    },
    {
        "partition_pct": 40,
        "bench_size": 20,
        "val_rows": len(partition_40["val"]),
        "test_rows": len(partition_40["test"]),
        "seconds_per_run": est_40_20["seconds_per_run"],
        "est_val_hours": est_40_20["estimated_full_val_seconds"] / 3600,
        "est_test_hours": est_40_20["estimated_full_test_seconds"] / 3600,
        "est_prompt_total_hours": est_40_20["estimated_prompt_total_seconds"] /
↪3600,
    },
]

comparison_df = pd.DataFrame(rows)
display(comparison_df.sort_values(by=["partition_pct", "bench_size"]).
↪reset_index(drop=True))

```

	partition_pct	bench_size	val_rows	test_rows	seconds_per_run \
0	20	10	159	159	4.592388
1	20	20	159	159	5.093507
2	30	10	238	238	4.594975
3	40	10	317	318	5.727517
4	40	20	317	318	5.085923

	est_val_hours	est_test_hours	est_prompt_total_hours
0	0.811322	0.405661	1.216983

1	0.899853	0.449926	1.349779
2	1.215116	0.607558	1.822673
3	2.017359	1.011861	3.029220
4	1.791375	0.898513	2.689888

- Bueno: val completo en menos de 2 horas
- Aceptable: entre 2 y 4 horas
- Pesado pero posible: entre 4 y 6 horas
- Malo para iterar: más de 6 horas solo para val